

Edge Detection

Edge Detection

1. Course Content
2. Start Agent
3. Camera Angle Adjustment
4. Run Case
5. Source Code Analysis

1. Course Content

- Run example programs, use depth images for edge detection. During movement, if an edge is detected, the car will stop at the edge to prevent the risk of falling.

2. Start Agent

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

Note: If already started, no need to restart

Enter command in the car terminal:

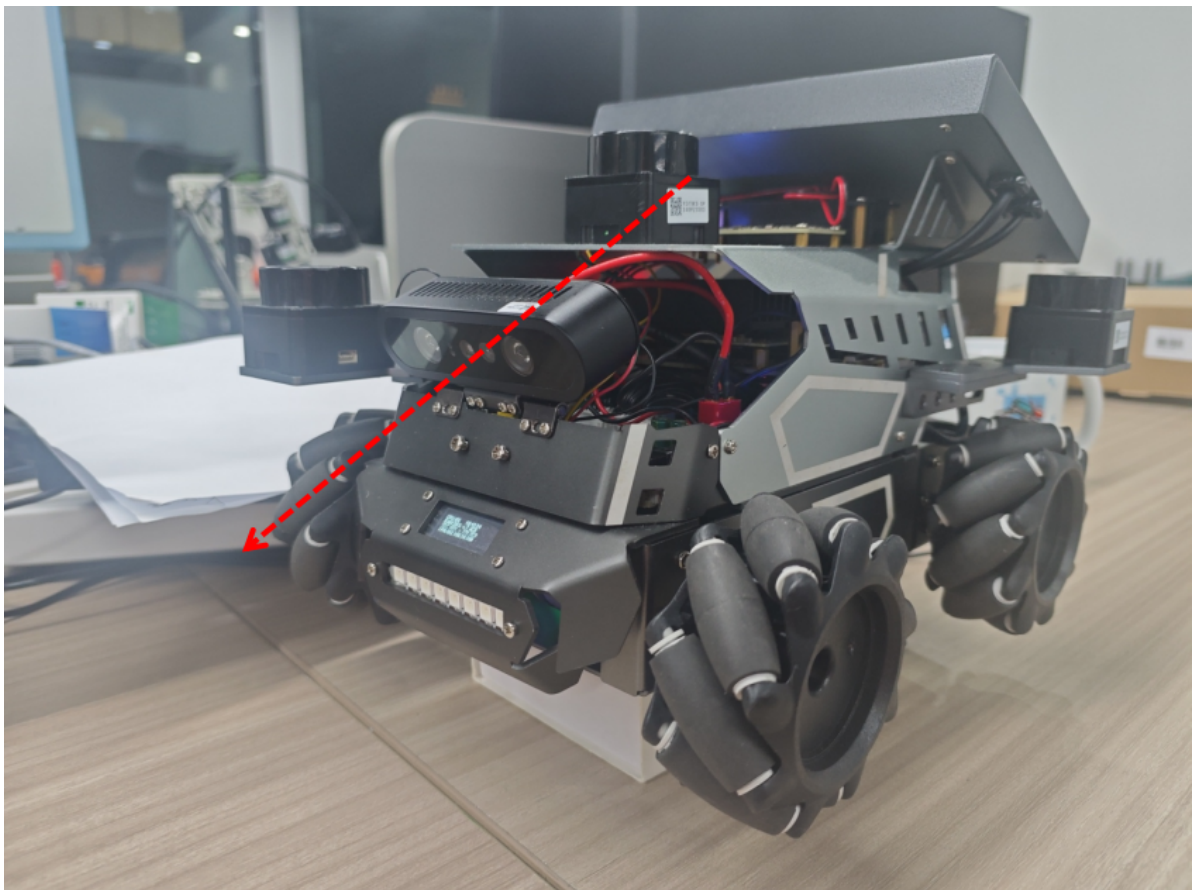
```
start_agent
```

The terminal prints the following information, indicating successful connection

```
mircoROS_Agent
[1764932423.553407] info | ProxyClient.cpp | create_datareader | d
subscriber created | client_key: 0x14F22590, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1764932423.557165] info | ProxyClient.cpp | create_topic | t
topic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_
id: 0x000(1)
[1764932423.560435] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1764932423.565261] info | ProxyClient.cpp | create_datareader | d
datareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1764932423.569631] info | ProxyClient.cpp | create_topic | t
topic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_
id: 0x000(1)
[1764932423.573464] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1764932423.578415] info | ProxyClient.cpp | create_datareader | d
datareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

3. Camera Angle Adjustment

- When running this case, it is best to adjust the camera angle downward slightly, so that the edge lines observed in the camera depth image are clearest, and the recognition effect will be better.

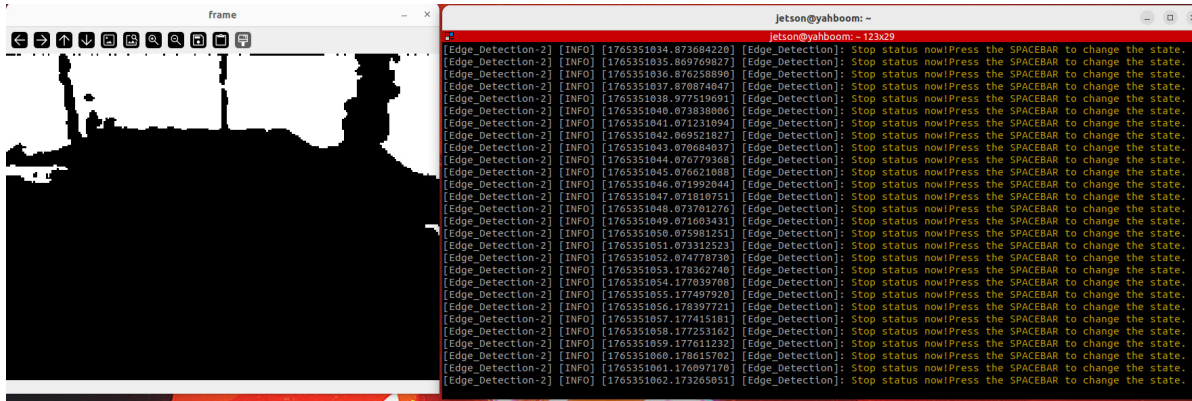


4. Run Case

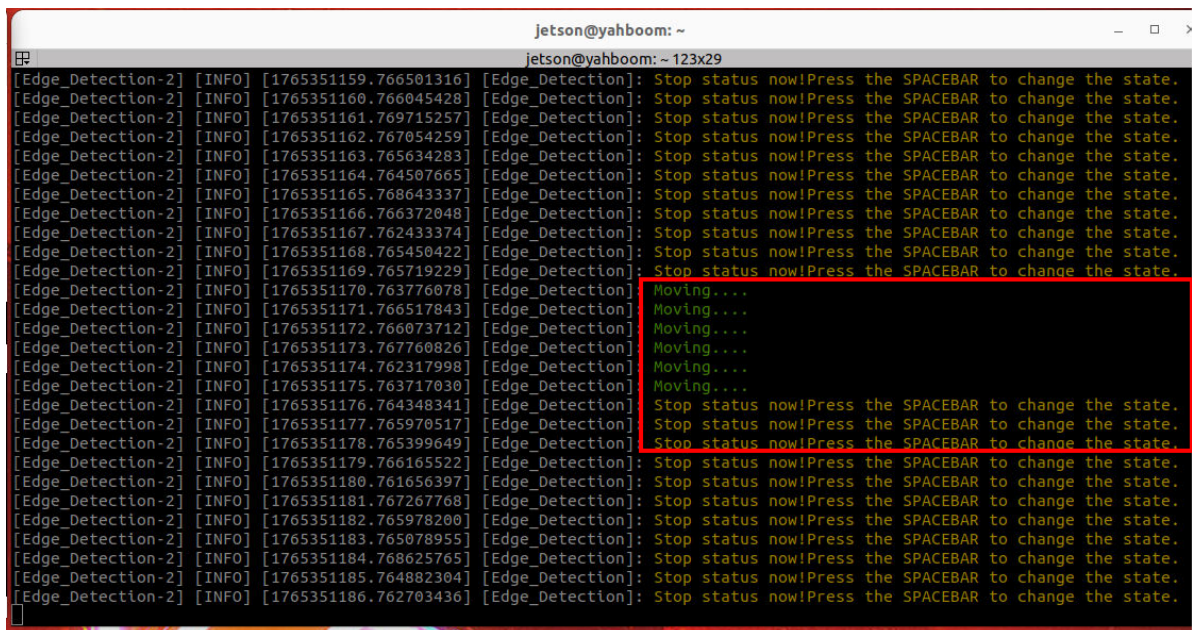
- Enter command in terminal:

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=Edge_Detection
```

- After the program starts, the initial state is stopped.
- After pressing the spacebar on the `frame` window, the car will start moving forward
- If no edge is detected, the car will move forward and print "Moving...."



- If an edge is detected, it will stop and print "Stop!!!".



5. Source Code Analysis

Program source code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/Edge_Detection.py
```

- Import necessary library files

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from geometry_msgs.msg import Twist
from arm_msgs.msg import ArmJoints
from cv_bridge import CvBridge
import cv2
import numpy as np
import threading
```

- Depth image decoding format

```
encoding = ['16UC1', '32FC1']
```

- Initialize variables and define publisher and subscriber,

```
def __init__(self, name):
    super().__init__(name)
    self.declare_parameter('depth_topic', '/camera/depth/image_raw')

depth_topic=self.get_parameter("depth_topic").get_parameter_value().string_value
self.pub_vel = self.create_publisher(Twist, '/cmd_vel',1)
self.sub_depth =
self.create_subscription(Image,depth_topic,self.get_DepthImageCallback,1)

self.lin_x = 0.1
self.Joy_active = False
self.depth_bridge = CvBridge()
self.move_flag = False
self.count=0
```

Depth image topic callback function, and calculate center point depth distance information,

```
def get_DepthImageCallback(self,msg):
    depth_image = self.depth_bridge.imgmsg_to_cv2(msg, encoding[1])
    #Call thread to pass the obtained depth image, calculate depth information
    compute_ = threading.Thread(target=self.compute_dist, args=(depth_image,))
    compute_.start()
    compute_.join()
    key = cv2.waitKey(10)
    if key == 32:
        self.move_flag = True
    cv2.imshow("frame", depth_image)

def compute_dist(self,result_frame):
    frame = cv2.resize(result_frame, (640, 480))
    depth_image_info = frame.astype(np.float32)
    #Determine if the current state is moving state, if so, determine the
    distance of the center point, if not, call function, publish stop command, and
    print information
    if self.move_flag == True:
```



```

        #Determine if the depth information of the center point, that is, the
        point x=320,y=240 is greater than 0.5m, if so, call function, publish stop
        command, if not, call function, publish forward command.
        if depth_image_info[240, 320]/1000>0.5:
            self.pubvel(0,0,0)
            self.move_flag = False
            print("Stop!!!")
        else:
            self.pubvel(self.lin_x,0,0)
            print("Moving....")
    else:
        self.pubvel(0,0,0)
        print("Stop status now!Press the SPACEBAR to change the state.")

```

- Publish velocity function, pass three variables, which are x-direction velocity, y-direction velocity and angular velocity, after assignment, call `self.pub_vel.publish(vel)` to publish velocity topic.

```

def pubvel(self,vx,vy,vz):
    vel = Twist()
    vel.linear.x = float(vx)
    vel.linear.y = float(vy)
    vel.angular.z = float(vz)
    self.pub_vel.publish(vel)

```